Лабораторная работа №3 «Визуализация поиска решений генетического алгоритма»

В лабораторной работе № 2 бало выполнено решение ГА для решения задачи OneMax (код программы преведен ниже). Рассмотрим визуализацию приспособленностей особей на каждой итерации (поколении).

ex1.py

```
from deap import base, algorithms
from deap import creator
from deap import tools

import random
import matplotlib.pyplot as plt
import numpy as np

# константы задачи
ONE_MAX_LENGTH = 100    # длина подлежащей оптимизации битовой строки

# константы генетического алгоритма
POPULATION_SIZE = 200   # количество индивидуумов в популяции
P_CROSSOVER = 0.9       # вероятность скрещивания
P_MUTATION = 0.1        # вероятность мутации индивидуума
MAX_GENERATIONS = 50    # максимальное количество поколений

RANDOM_SEED = 42
random.seed(RANDOM_SEED)

creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

def oneMaxFitness(individual):
    return sum(individual), # кортеж

toolbox = base.Toolbox()

toolbox.register("zeroOrOne", random.randint, 0, 1)
toolbox.register("individualCreator", tools.initRepeat, creator.Individual, toolbox.zeroOrOne, ONE_MAX_LENGTH)
toolbox.register("populationCreator", tools.initRepeat, list, toolbox.individualCreator)

population = toolbox.populationCreator(n=POPULATION_SIZE)

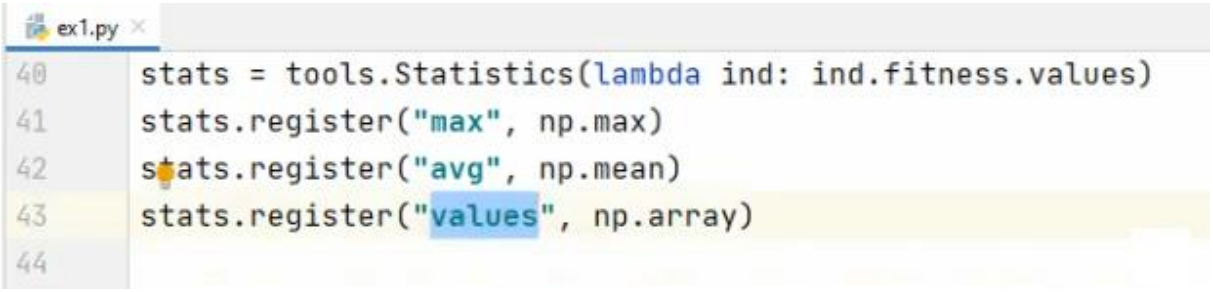
toolbox.register("evaluate", oneMaxFitness)
toolbox.register("select", tools.selTournament, tournsize=3)
toolbox.register("mate", tools.cxOnePoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/ONE_MAX_LENGTH)

stats = tools.Statistics(lambda ind: ind.fitness.values)
stats.register("max", np.max)
stats.register("avg", np.mean)
```

```
39
40  stats = tools.Statistics(lambda ind: ind.fitness.values)
41  stats.register("max", np.max)
42  stats.register("avg", np.mean)
43
44  population, logbook = algorithms.eaSimple(population, toolbox,
45                                            cxpb=P_CROSSOVER,
46                                            mutpb=P_MUTATION,
47                                            ngen=MAX_GENERATIONS,
48                                            stats=stats,
49                                            verbose=True)
50
51  maxFitnessValues, meanFitnessValues = logbook.select("max", "avg")
52
53  plt.plot(maxFitnessValues, color='red')
54  plt.plot(meanFitnessValues, color='green')
55  plt.xlabel('Поколение')
56  plt.ylabel('Макс/средняя приспособленность')
57  plt.title('Зависимость максимальной и средней приспособленности от поколения')
58  plt.show()
59
```

Вначале у статистки зарегистрируем еще один параметр:

stats.register("values", np.array)



values будет хранить список значений приспособленности особей на текущей итерации, а затем будем его хранить в коллекции **vals** (добавляем эту запись)**.**

maxFitnessValues, meanFitnessValues, vals = logbook.select("max", "avg", "values")

```
ex1.py
40  stats = tools.Statistics(lambda ind: ind.fitness.values)
41  stats.register("max", np.max)
42  stats.register("avg", np.mean)
43  stats.register("values", np.array)
44
45  population, logbook = algorithms.eaSimple(population, toolbox,
46                                            cxpb=P_CROSSOVER,
47                                            mutpb=P_MUTATION,
48                                            ngen=MAX_GENERATIONS,
49                                            stats=stats,
50                                            verbose=True)
51
52  maxFitnessValues, meanFitnessValues, vals = logbook.select("max", "avg", "values")
53
```

Далее вместо простого вывода графиков указываем интерактивное.

```
53
54  plt.plot(maxFitnessValues, color='red')
55  plt.plot(meanFitnessValues, color='green')
56  plt.xlabel('Поколение')
57  plt.ylabel('Макс/средняя приспособленность')
58  plt.title('Зависимость максимальной и средней приспособленности от поколения')
59  plt.show()
```

```
53
54  # plt.plot(maxFitnessValues, color='red')
55  # plt.plot(meanFitnessValues, color='green')
56  # plt.xlabel('Поколение')
57  # plt.ylabel('Макс/средняя приспособленность')
58  # plt.title('Зависимость максимальной и средней приспособленности от поколения')
59  # plt.show()
60
```

Для этого создадим цикл по коллекции vals. На каждой итерации будем формировать график с текущими значениями приспособленности каждой особи. Более подробно о создании анимации в пакете Matplotlib можно узнать https://youtu.be/YWt3SKpZwKM

```
import time

plt.ion()
fig, ax = plt.subplots()

line, = ax.plot(vals[0], ' o', markersize=1)
ax.set_ylim(40, 110)

for v in vals:
    line.set_ydata(v)

    plt.draw()
    plt.gcf().canvas.flush_events()

    time.sleep(0.5)

plt.ioff()
plt.show()
```

```python
maxFitnessValues, meanFitnessValues, vals = logbook.select("max", "avg", "values")

import time

plt.ion()
fig, ax = plt.subplots()

line, = ax.plot(vals[0], ' o', markersize=1)
ax.set_ylim(40, 110)

for v in vals:
    line.set_ydata(v)

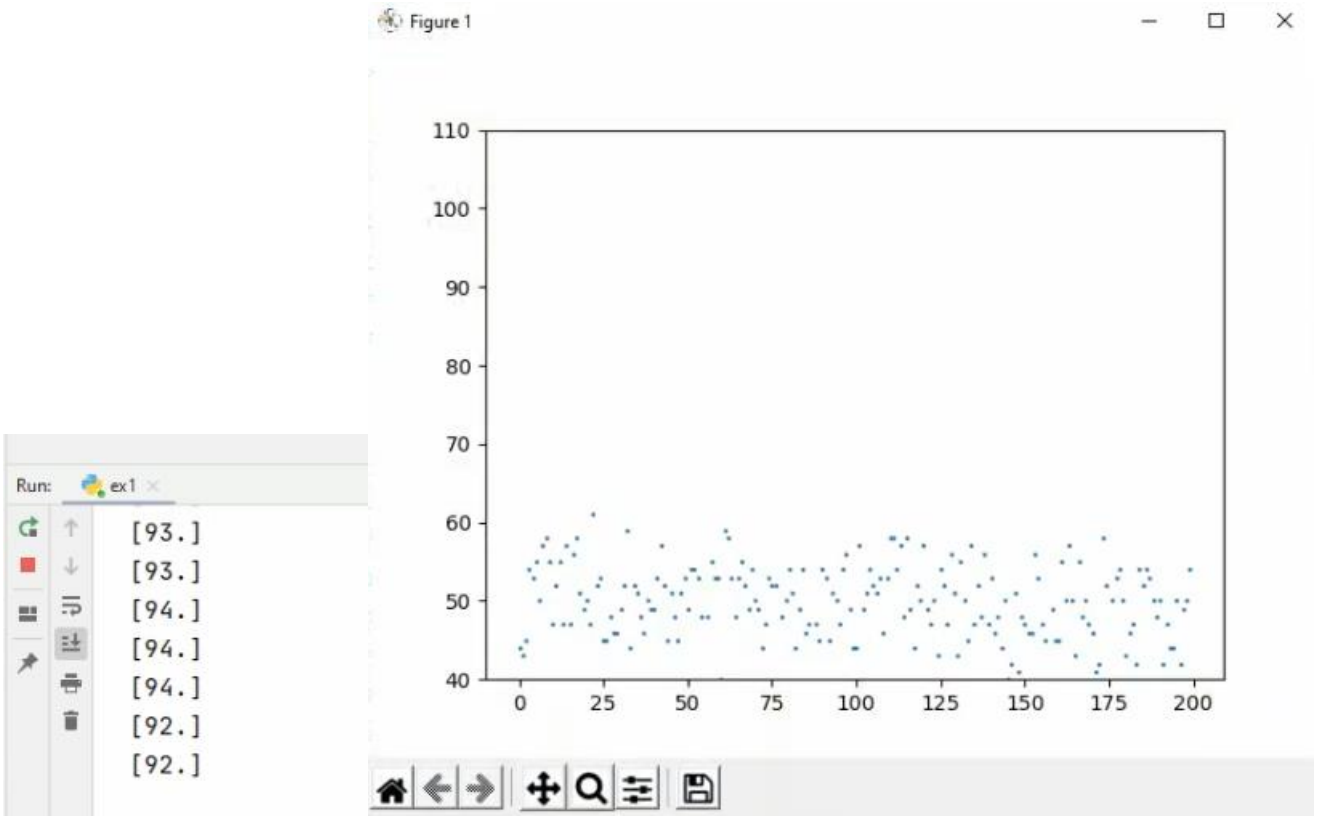
    plt.draw()
    plt.gcf().canvas.flush_events()

    time.sleep(0.5)

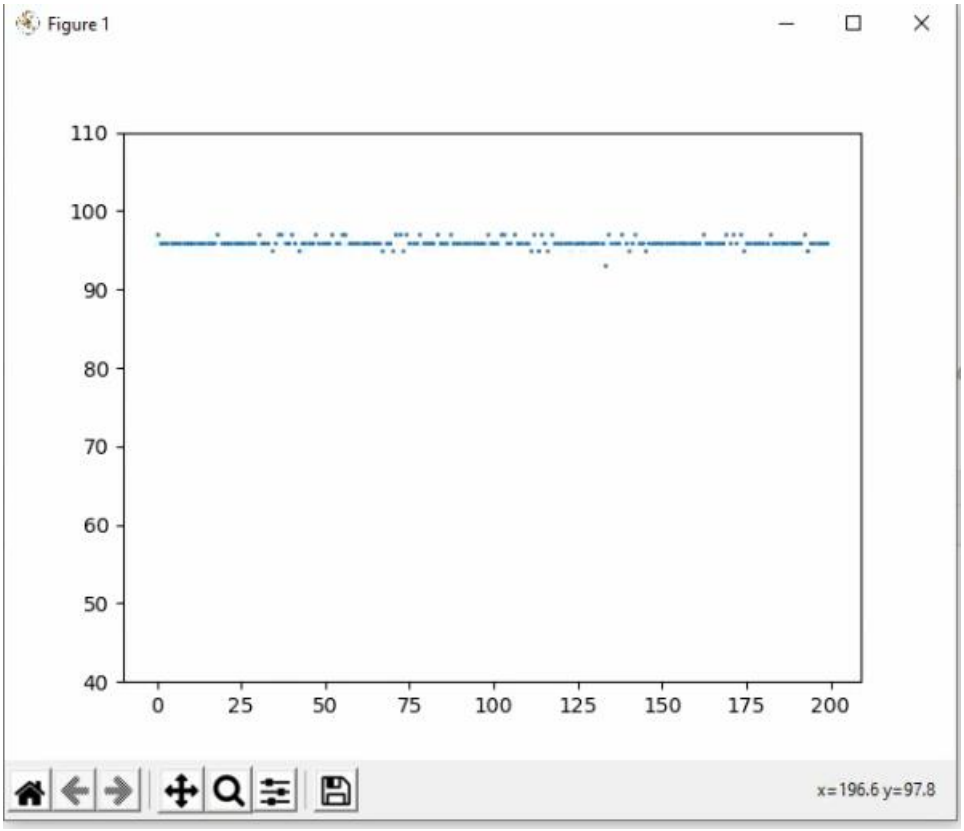
plt.ioff()
plt.show()

# plt.plot(maxFitnessValues, color='red')
```

После запуска программы мы увидим.



Каждая точка есть значение функции приспособленности отдельного индивидуума в текущей поколении. От кадра к кадру происходит смена поколений и в целом популяция стремится к уровню 100 (максимальному значению приспособленности особей).



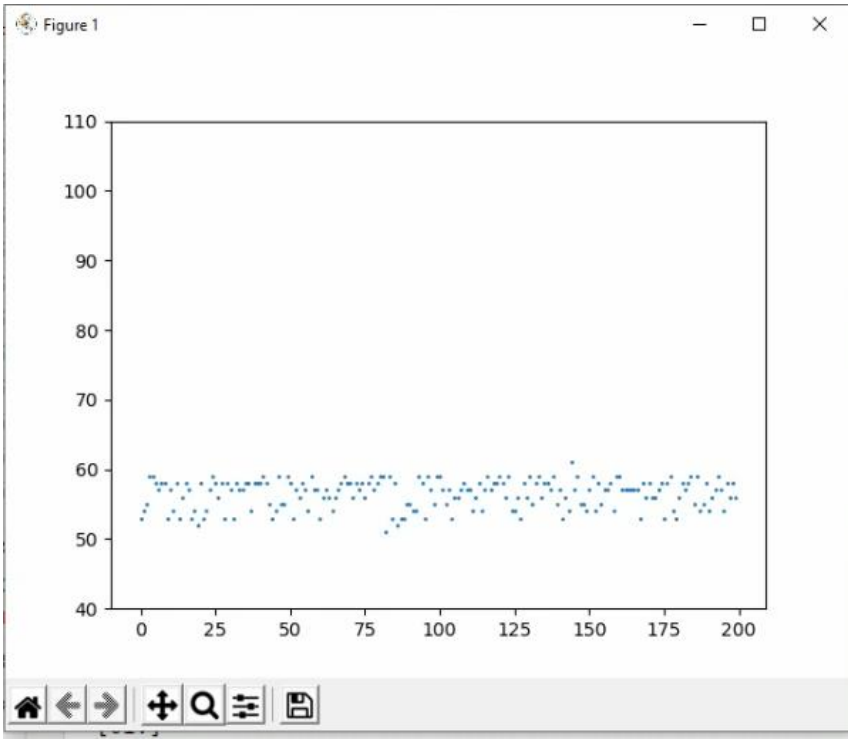
Рассмотрим подробно процесс эволюции, заложенный в ГА. Движущая сила эволюции – это естественный отбор. В нашем алгоритме за этот естественный отбор отвечает функция турнирного отбора (функция **select**).

```
35  toolbox.register("evaluate", oneMaxFitness)
36  toolbox.register("select", tools.selTournament, tournsize=3)
37  toolbox.register("mate", tools.cxOnePoint)
```

Если в алгоритме отключим скрещивание и мутацию, т.е. поставим вероятность равную 0.

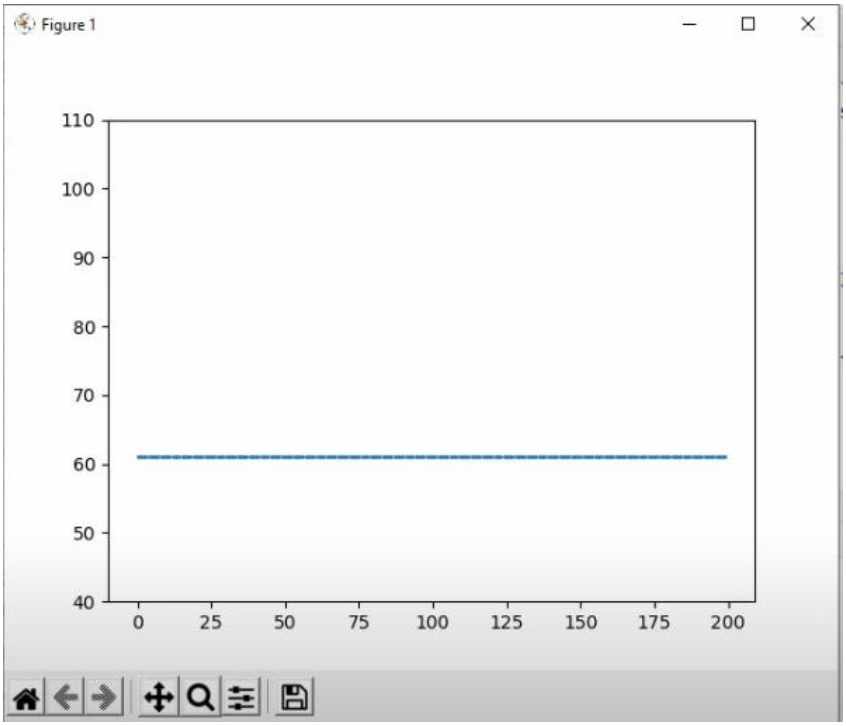
```
40  stats = tools.Statistics(lambda ind: ind.fitness.values)
41  stats.register("max", np.max)
42  stats.register("avg", np.mean)
43  stats.register("values", np.array)
44
45  population, logbook = algorithms.eaSimple(population, toolbox,
46                                             cxpb=0, #P_CROSSOVER,
47                                             mutpb=0, #P_MUTATION,
48                                             ngen=MAX_GENERATIONS,
49                                             stats=stats,
50                                             verbose=True)
```

Будет происходить только естественный отбор. Запустим программу. Видно, что особи улучшаются.



Через несколько поколений они становятся идентичны друг другу. В процессе отбора худшие особи отсеялись и остались только лучшие. В итоги, все популяция устремилась к лучшему индивидууму, который был в начальной популяции. Дольше процесс эволюции не пошел. Популяция не стала улучшаться, и достигать максимального значения (100), так как хромосомы в

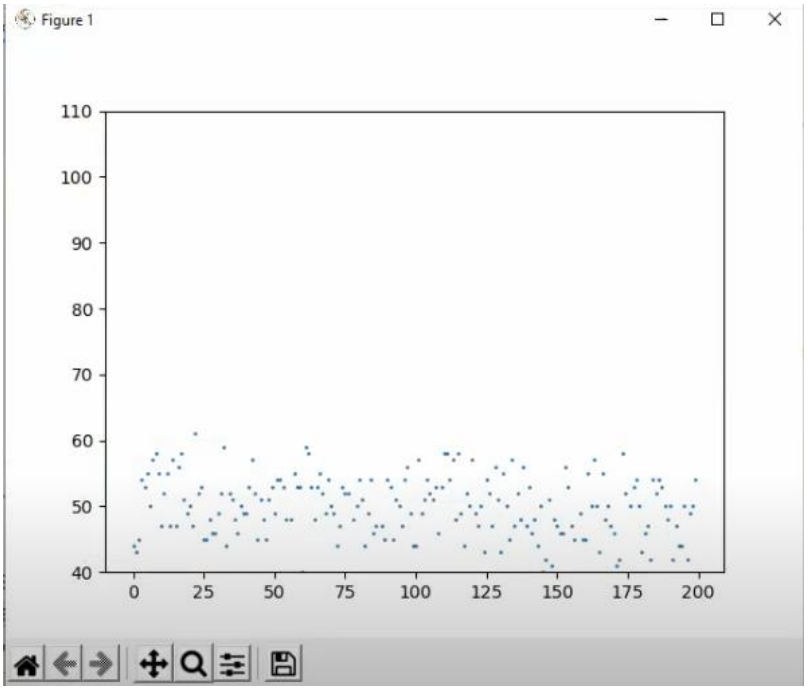
начальной популяции не меняются. Отбрасывается в данном случае только худшее. Это ведет к утрачиванию разнообразия особей, и все становятся идентичны друг другу.



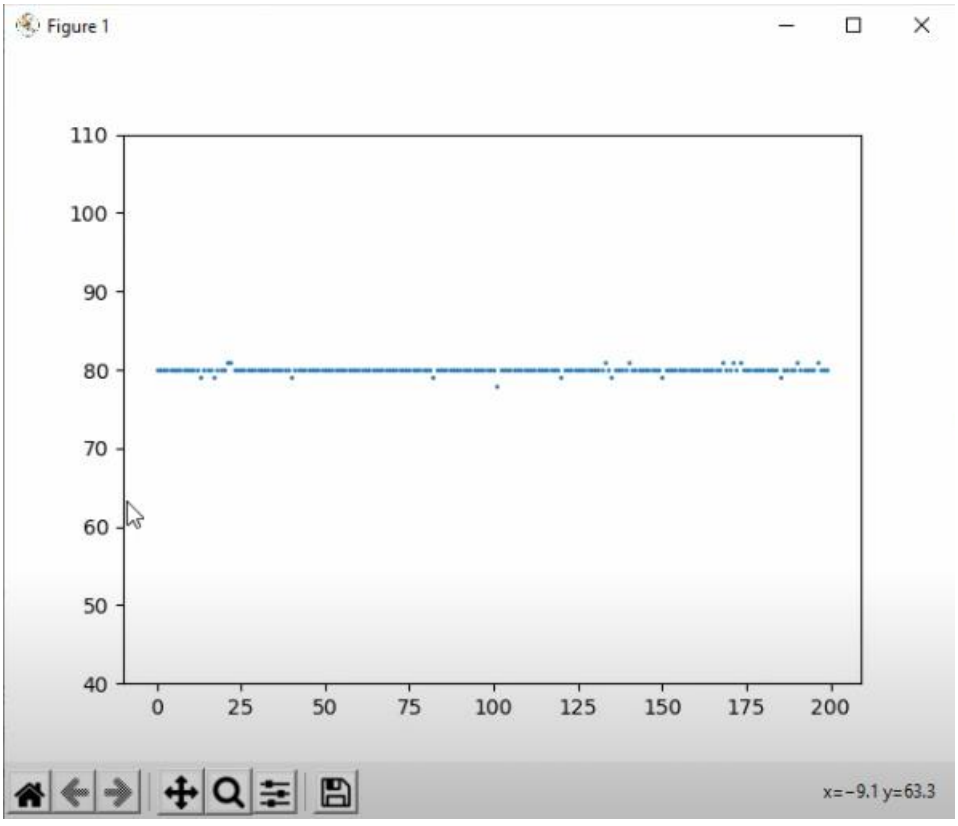
Рассмотрим, как изменится работа алгоритма, если добавим мутации. Для этого внесем изменения в программу (вернем мутации).

```
40  stats = tools.Statistics(lambda ind: ind.fitness.values)
41  stats.register("max", np.max)
42  stats.register("avg", np.mean)
43  stats.register("values", np.array)
44
45  population, logbook = algorithms.eaSimple(population, toolbox,
46                                             cxpb=0, #P_CROSSOVER,
47                                             mutpb=P_MUTATION,
48                                             ngen=MAX_GENERATIONS,
49                                             stats=stats,
50                                             verbose=True)
```

Запустим программу. Это изменения вносит разнообразие в программу.



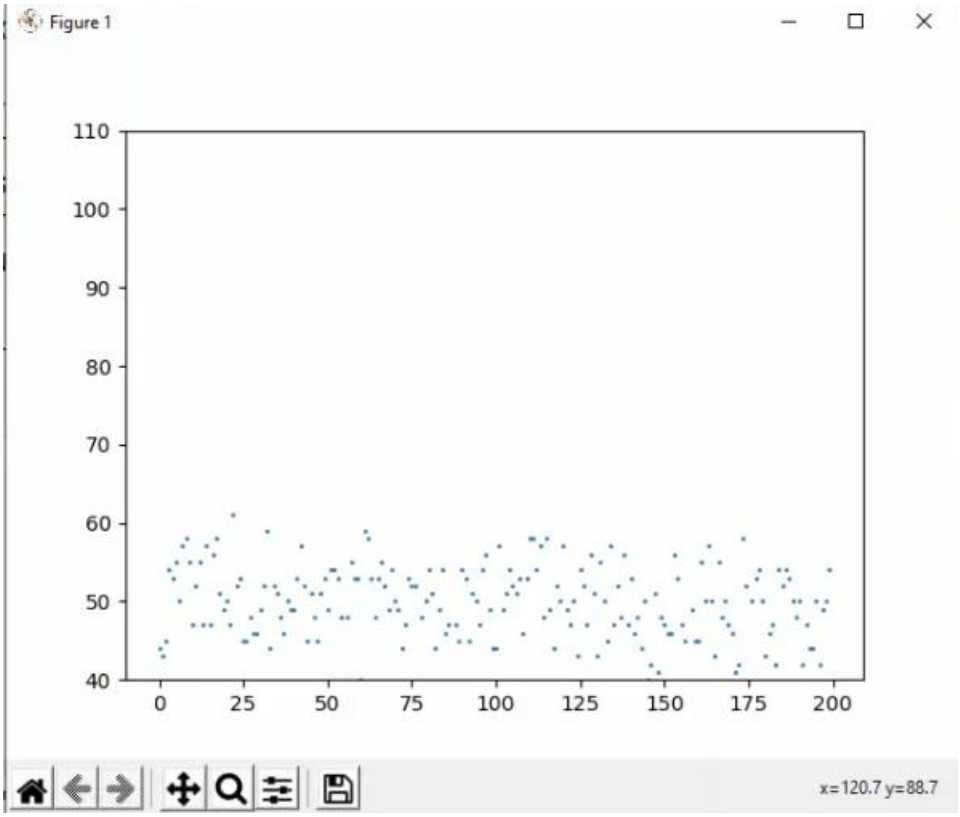
Время от времени, появляются лучшие особи, и оператор отбора подгоняет остальных особей вверх. Популяция становится лучше. Однако, после итерации в 50 поколений лучшее решение не будет достигнуто (100). Развитие остановилось на уровни 80.



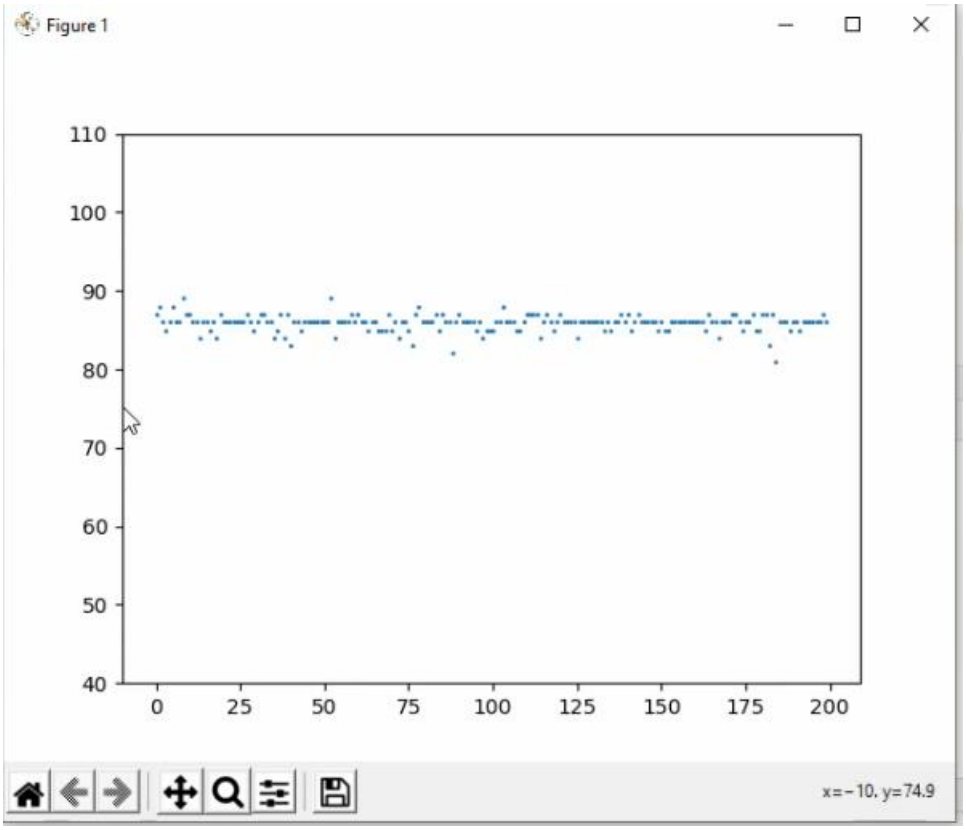
Давайте посмотрим, как влияет уровень мутации. Для этого программе вероятность мутации поставим 0,5.

```
11
12  # константы генетического алгоритма
13  POPULATION_SIZE = 200   # количество индивидуумов в популяции
14  P_CROSSOVER = 0.9       # вероятность скрещивания
15  P_MUTATION = 0.5        # вероятность мутации индивидуума
16  MAX_GENERATIONS = 50    # максимальное количество поколений
17
```

Запустим программу. Теперь разнообразие в популяции стало больше. Вероятность мутации увеличилась. Изменения в генах происходит чаще. Особи отличаются большим разнообразием. Чаще будут появляться лучшее особи, и турнирный отбор их будет выбирать, а остальная популяция будет подтягиваться к этим лучшим особям.



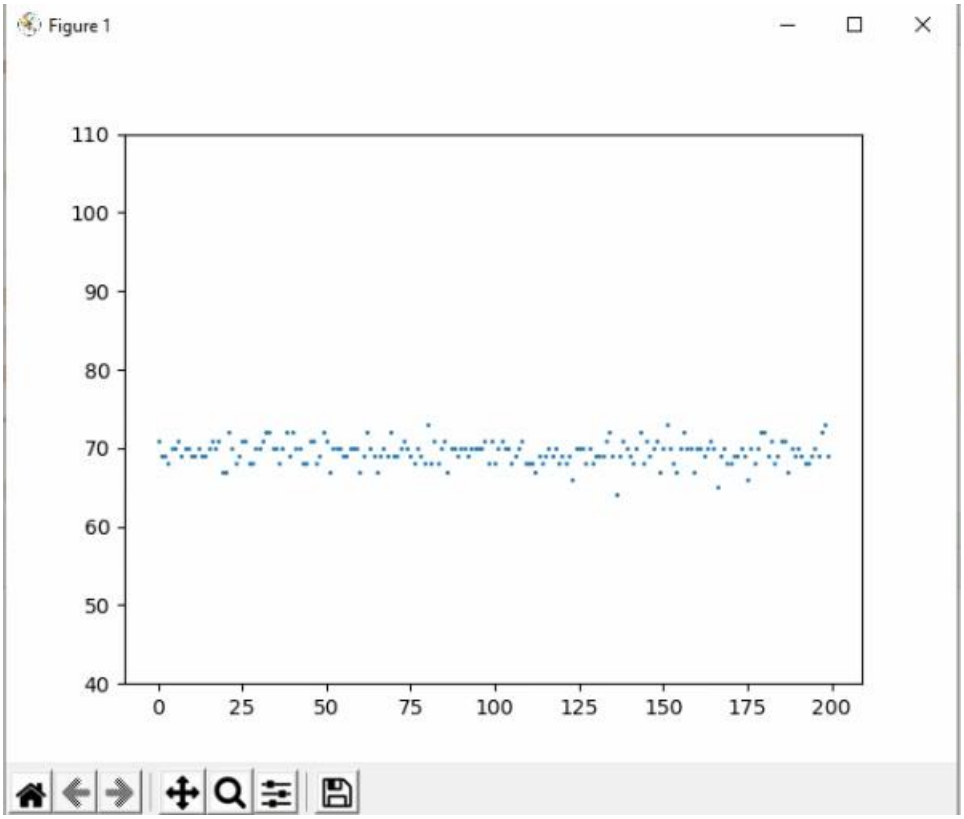
Теперь достигли более высоких значений – это 90.

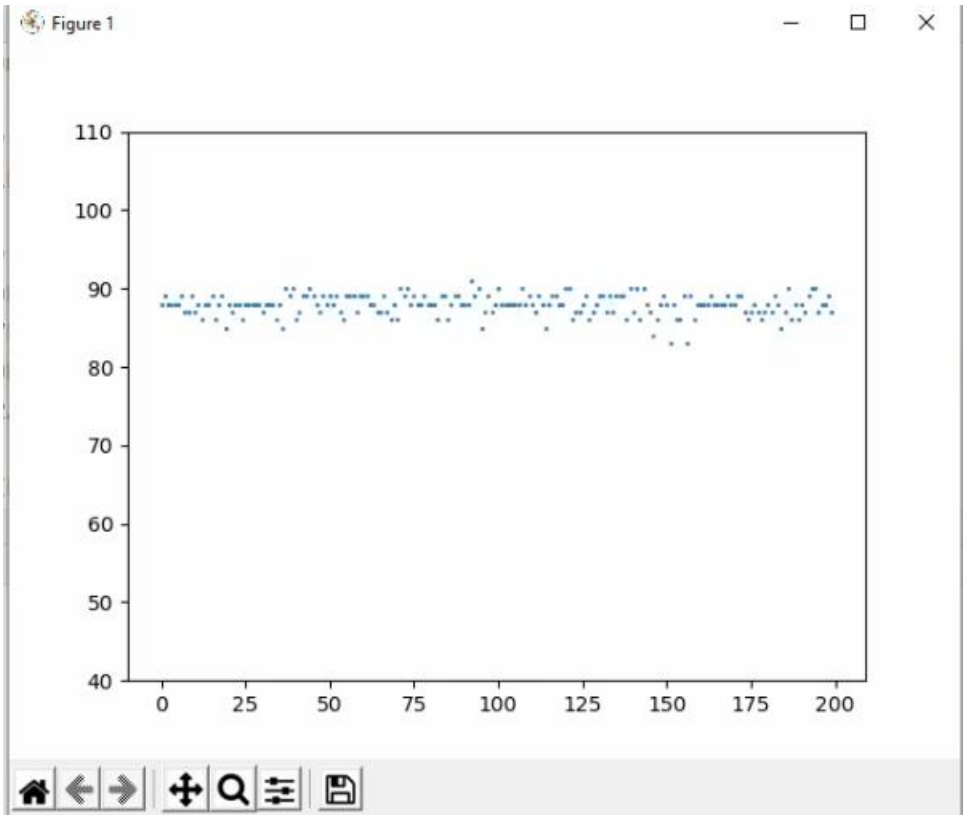


Попробуем изменить вероятность мутации до 0.9.

```
12  # константы генетического алгоритма
13  POPULATION_SIZE = 200   # количество индивидуумов в популяции
14  P_CROSSOVER = 0.9       # вероятность скрещивания
15  P_MUTATION = 0.9        # вероятность мутации индивидуума
16  MAX_GENERATIONS = 50    # максимальное количество поколений
17
```

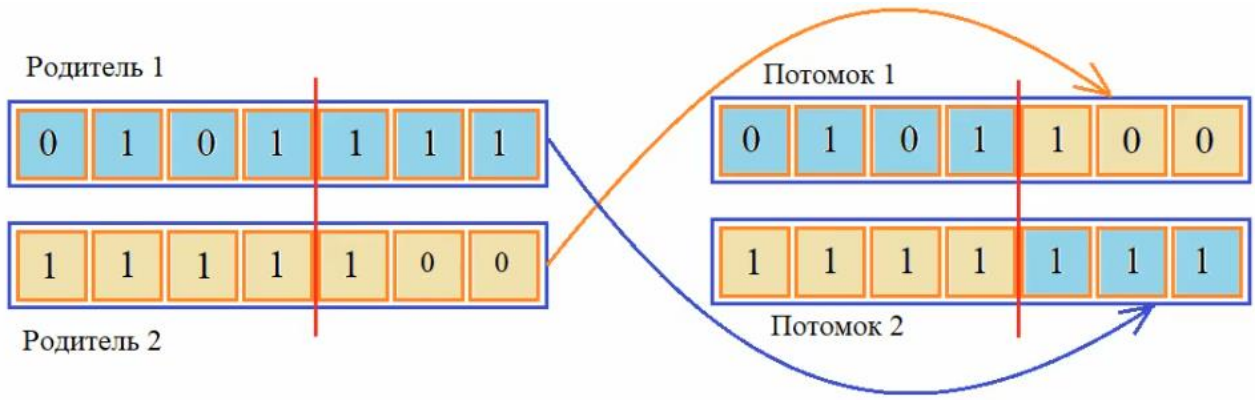
Запустим программу и посмотрим, как это повлияет на работу ГА.





После 50 итераций дошли до уровня, примерно 90, т.е. до максимального уровня в 100 дойти не получилось.

Таким образом, ГА – это не просто случайный направленный поиск. Важную роль играет оператор кроссинговера (скрещивания), когда части хромосом родителей перемешиваются для получения потомков.

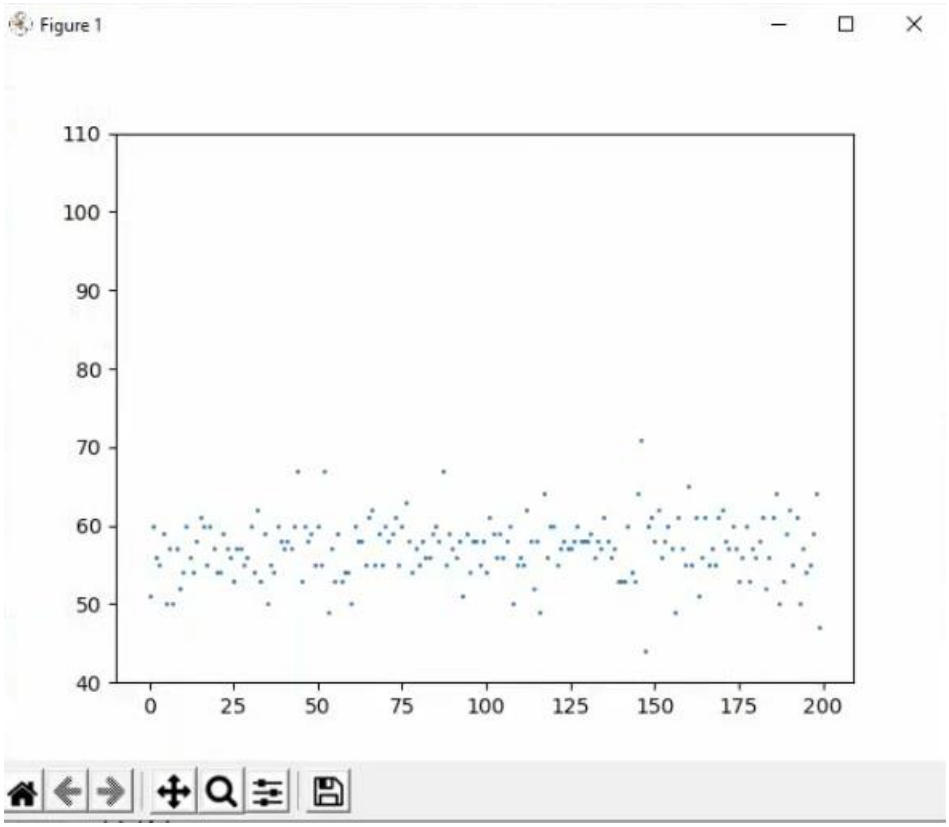


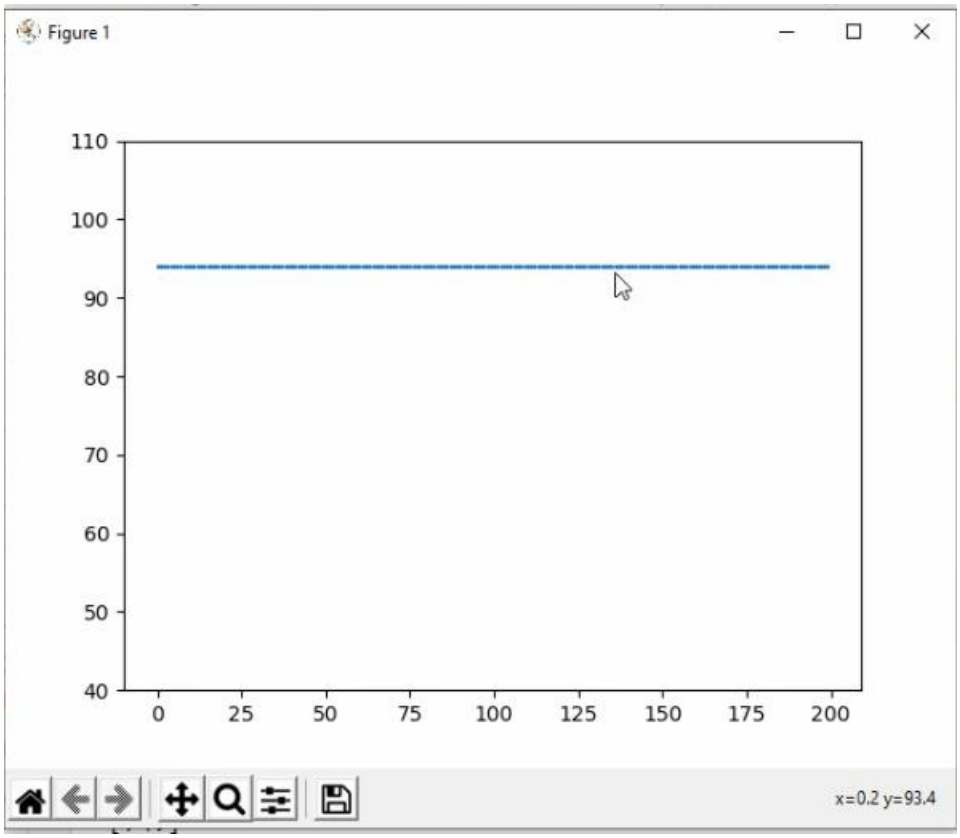
Это работает гораздо эффективнее простой мутации. Представьте, что в двух разных частях хромосом родителей уже сформировано оптимальное решение. Тогда достаточно произвести обмен и будет получен потомок с наилучшим набором генов. Именно благодаря этому операция кроссинговера дает заметное ускорение сходимости ГА к оптимальному решению.

Вернем скрещивание в нашу программу, а мутацию отключим.

```
44
45  population, logbook = algorithms.eaSimple(population, toolbox,
46                                             cxpb=P_CROSSOVER,
47                                             mutpb=0, #P_MUTATION,
48                                             ngen=MAX_GENERATIONS,
49                                             stats=stats,
50                                             verbose=True)
```

Запустим алгоритм (без мутации, но со скрещиванием). Популяция довольно быстро поднимается вверх. Через несколько итераций она останавливается на уровни 97. Максимальное значение не достигается.





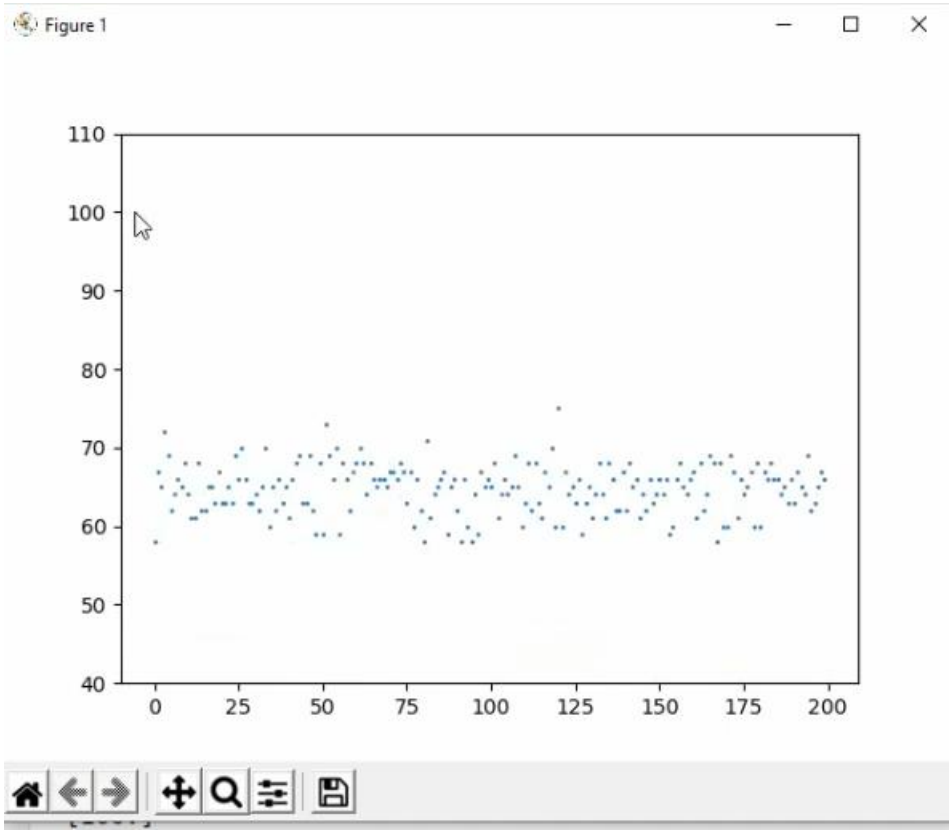
Отбор и скрещивание дают лучший результат, чем отбор и мутация. В процессе скрещивания алгоритм пытается получить лучших потомков, ему это

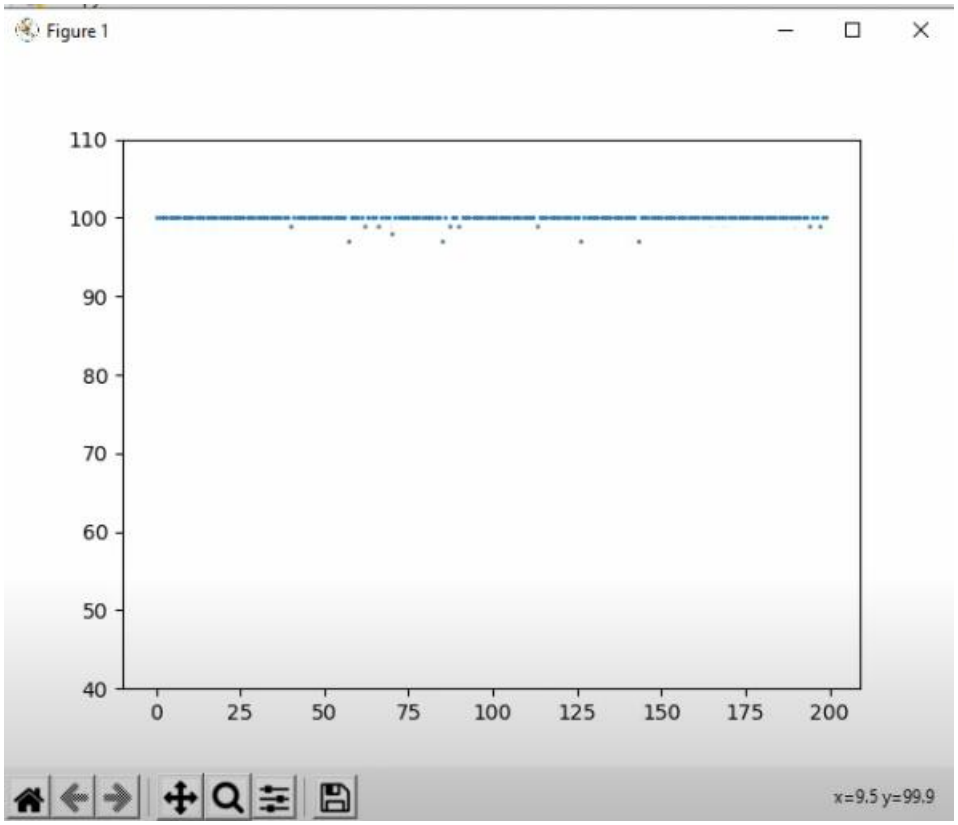
удается. После этапа отбора, он оставляет лучших потомков, т.е. решение постоянно улучшается. Однако, уровня в 100 не достигли. В процессе скрещивания и отбора были потеряны особи, которые хранили нужные гены для достижения оптимального решения. Мутация решает эту проблему. Она вносит небольшое разнообразие в популяцию и случайно потерянная информация (фрагменты хромосом) - случайно, благодаря мутации восстанавливаются. Это гарантирует при большом числе итераций существование достаточных данных для нахождения оптимального решения.

Возвращаем мутацию с вероятностью 0.1 – получим прежний алгоритм. ГА будет находить решение на 100. Получилась популяция дающая лучшее решение.

```
44
45  population, logbook = algorithms.eaSimple(population, toolbox,
46                                             cxpb=P_CROSSOVER,
47                                             mutpb=P_MUTATION,
48                                             ngen=MAX_GENERATIONS,
49                                             stats=stats,
50                                             verbose=True)
```

```
13  POPULATION_SIZE = 200    # количество индивидуумов в популяции
14  P_CROSSOVER = 0.9        # вероятность скрещивания
15  P_MUTATION = 0.1         # вероятность мутации индивидуума
16  MAX_GENERATIONS = 50     # максимальное количество поколений
17
```





Таким образом, три главных компонента – отбор, скрещивание, мутация позволяют получить лучшее решение.

Преимущества и недостатки ГА

Как у всех алгоритм, ГА имеют свои преимущества и недостатки по сравнению с другими алгоритмами оптимизации (назовем их классическими).

Преимущества генетических алгоритмов:

- способны находить глобальные оптимумы (с большей вероятностью, чем традиционные алгоритмы);
- можно применять в задачах, которые сложно формализовать математически, либо даже совсем не имеющие математического описания (функция приспособленности может быть выражена на уровне алгоритма);
- устойчивы к шуму (например, когда данные не могут быть точно измерены или основаны на субъективном восприятии человека);
- можно реализовать на уровне параллельных вычислений для ускорения работы алгоритма;
- возможность непрерывного обучения (оптимизации) популяции при постоянном изменении окружающей среды (это изменение сказывается на значениях функции принадлежности и популяции необходимо постоянно подстраиваться под новые обстоятельства).

Недостатки генетических алгоритмов

К общим недостаткам при реализации ГА можно отнести следующие моменты:

- специфичная формализация исходной задачи на уровне генов и хромосом, а также операторов отбора, скрещивания и мутации;
- определение внешних параметров работы алгоритма (размер популяции, вероятности скрещивания и мутации, способ отбора и кроссинговера и т.п.);
- относительно большой объем вычислений (каждая особь в популяции, фактически, отдельный ход оптимизации задачи);
- некоторые особи в популяции могут быстро занять локальный оптимум и потянуть за собой всю популяцию (будут продублированы оператором отбора много раз, и популяция будет состоять, в основном, из их отпрысков), в результате другой, глобальный оптимум может быть не обнаружен;
- генетический алгоритм не гарантирует нахождение именно оптимального решения, нам приходится довольствоваться тем лучшим решением, которое было найдено (однако, это недостаток многих алгоритмов оптимизации).